CRM Database Guide

Oct 19, 2026 · @R Kishore Reddy

The CRM stores everything in 27 PostgreSQL tables in the `public` schema of the Neon database `neondb`. A lead is the centre of the design: every other business table either describes it, records work done on it, or sits beside it.

The SQL lives in `backend/schema.sql` and is safe to re-run. All access is plain SQL through psycopg, with no ORM, so this guide is also the map for writing queries.

The 27 tables in 8 groups

Group	Tables	What the group is for
Lookups (9)	roles, lead_sources, pipeline_stages, lost_reasons, activity_types, call_outcomes, revision_types, revision_statuses, tech_stacks	Small editable lists that fill dropdowns
People (3)	users, companies, contacts	Your team, the businesses you sell to, and the people there
Leads (3)	leads, lead_stage_history, import_batches	The deal itself, its stage trail, and Excel uploads
Work on a lead (2)	activities, follow_ups	Call summaries and reminders
Requirements (3)	requirements, requirement_tech_stacks, requirement_features	What the client wants built
Prototype loop (2)	prototype_revisions, revision_feedback	Every round of change requests
Meetings (3)	meeting_series, meetings, meeting_attendees	Calendar and recurring meetings
Support (2)	attachments, email_log	File references and a record of sent emails

Conventions used by every table

- Each table has an `id` primary key (auto-generated), plus `created_at` and `updated_at`. A trigger refreshes `updated_at` on every update.
- A column ending in `_id` points to another table. Example: `leads.company_id` points to `companies.id`.
- Times are stored in UTC (`timestamptz`) and shown in IST by the app.
- Fixed choices that rarely change use a `CHECK` list. Choices your founders may want to edit use a lookup table.

How the tables connect

Every table hangs off the leads table

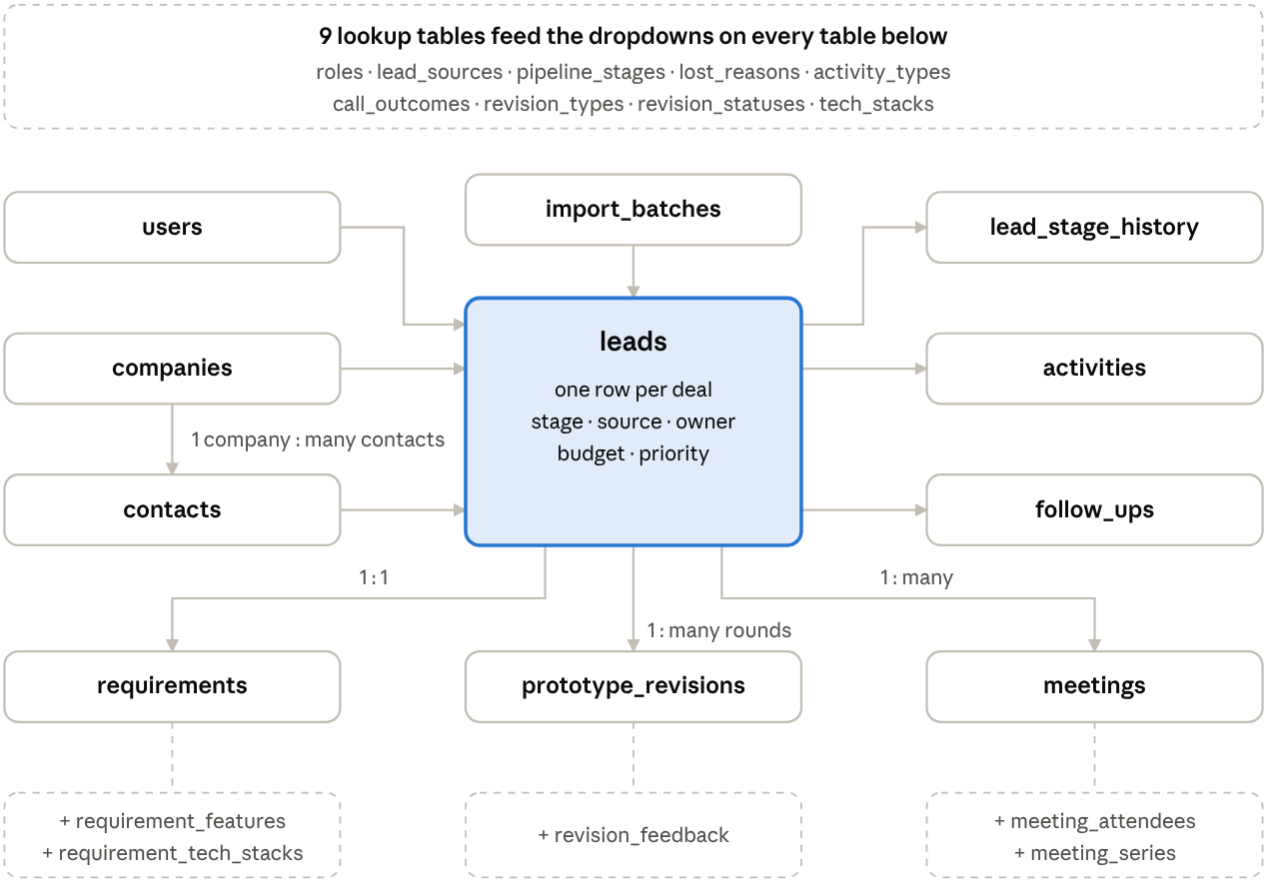


table relationships · 16 tables shown, arrows point from parent to child

Read each arrow as parent to child: one company has many contacts, and one lead has many activities, follow-ups and prototype rounds. The dashed boxes hold the child tables that hang off the table above them. Two more tables, `attachments` and `email_log`, also point at `leads` and are covered in the support section.

Lookup tables

Lookup tables are short lists that fill dropdowns. Each has `id`, `name` (unique) and the standard timestamps. Other tables store the `id`, never the word, so renaming a value here updates it everywhere.

Table	Seeded values	Used by
roles	founder, manager, intern	users.role_id, pipeline_stages.owner_role
lead_sources	call, whatsapp, excel, manual, other	leads.source_id
pipeline_stages	new, contacted, qualified, prototype, proposal, negotiation, won, lost	leads.stage_id, lead_stage_history
lost_reasons	budget too high, no response, chose competitor, requirements changed, other	leads.lost_reason_id
activity_types	call, whatsapp, email, note	activities.type_id
call_outcomes	answered, no answer, busy, interested, not interested, callback requested	activities.outcome_id
revision_types	initial, change request, new feature, bug fix	prototype_revisions.type_id
revision_statuses	planned, in progress, presented, approved, changes requested	prototype_revisions.status_id
tech_stacks	React, Next.js, Django, Node.js, Flutter, React Native, PostgreSQL, MySQL, MongoDB	requirement_tech_stacks

`pipeline_stages` is the only lookup with extra columns:

Column	Type	Meaning
position	integer, unique	Order on the kanban board (1 = new, 8 = lost)
owner_role	roles.id	Which role normally handles a lead at this stage. Drives handover.

People tables

Three tables answer "who": your team, the businesses you sell to, and the individuals at those businesses.

users

Your 5 team members. Login is email plus password. The password is stored hashed, never as plain text.

Column	Type	Meaning
id	bigint, PK	Identifier
email	text, unique	Login name
password	text	Hash of the password
full_name	text	Display name
phone	text, optional	Contact number, not used for login
role_id	roles.id	founder, manager or intern
is_active	boolean	Set false to disable someone while keeping their history

companies

The business that wants something built. One company can have several leads over time.

Column	Type	Meaning
id	bigint, PK	Identifier
name	text	Company name
website, industry, location	text, optional	Context for the team
created_by	users.id	Who added it

contacts

The people at a company. Exactly one should be marked primary.

Column	Type	Meaning
id	bigint, PK	Identifier
company_id	companies.id	Owning company. Deleting the company deletes its contacts.
name, job_title	text	Who they are
email, phone, alt_phone	text, optional	How to reach them
is_primary	boolean	The main person for calls and emails

Indexes on `lower(email)` and `phone` make duplicate checks fast when interns import leads.

Lead tables

leads

The central table. One row is one deal with one company. It holds the deal's current state; the history lives in other tables.

Column	Type	Meaning
id	bigint, PK	Identifier
company_id	companies.id	Who the deal is with
primary_contact_id	contacts.id, optional	The main person to talk to
source_id	lead_sources.id	Where the lead came from
stage_id	pipeline_stages.id	Current stage, from new to lost
project_summary	text	What they want built, in a few lines
estimated_budget	numeric	Rough budget
priority	high / medium / low	How important the lead is, set by the team
created_by	users.id	Intern who entered it
assigned_to	users.id	Current owner. Changes on handover.

Column	Type	Meaning
import_batch_id	import_batches.id, optional	Which Excel upload created it
lost_reason_id	lost_reasons.id, optional	Required in the app when stage is lost
deal_value	numeric, optional	Final value when won
stage_changed_at	timestamp	When the stage last changed, for time-in-stage reports
closed_at	timestamp, optional	When it became won or lost

lead_stage_history

One row every time a lead moves stage. The dashboard funnel and "how long did leads stay in each stage" both come from here, because `leads.stage_id` only knows where a lead is now.

Column	Type	Meaning
lead_id	leads.id	Which lead. Deleted with the lead.
from_stage	pipeline_stages.id, optional	Empty for the first row of a new lead
to_stage	pipeline_stages.id	The new stage
changed_by	users.id	Who moved it
changed_at	timestamp	When
note	text, optional	Reason for the move

import_batches

One row per Excel or CSV upload, so a bad import can be traced and undone.

Column	Type	Meaning
file_name	text	Uploaded file
uploaded_by	users.id	Who uploaded it
total_rows, imported_rows, skipped_rows	integer	Counts from the run
error_details	text, optional	Which rows failed and why

Work on a lead

activities

The timeline of everything that happened on a lead: calls, WhatsApp chats, emails and notes. The call summary your founders write after each call goes in `summary`.

Column	Type	Meaning
lead_id	leads.id	Which lead
contact_id	contacts.id, optional	Which person was spoken to
user_id	users.id	Who did it
type_id	activity_types.id	call, whatsapp, email or note
outcome_id	call_outcomes.id, optional	Answered, no answer, interested and so on
summary	text	What was discussed
occurred_at	timestamp	When it happened. Newest first in the app.

follow_ups

Reminders that make sure no lead goes quiet. A founder schedules one at the end of a call; the system emails the assignee when it is due.

Column	Type	Meaning
lead_id	leads.id	Which lead
assigned_to	users.id	Who must do it
due_at	timestamp	When it is due
purpose	text	Why, for example "share the requirements form"
status	pending / done / cancelled	Current state
completed_at	timestamp, optional	When marked done
reminder_sent_at	timestamp, optional	Set after the email goes out so it is never sent twice

A partial index on `due_at` covers pending rows only, so the "overdue and due today" list stays fast as history grows.

Requirements tables

After a client agrees to share requirements, the team records them here. A lead has at most one `requirements` row; the tech stack and the feature list hang off it as separate tables so you can later count "how many leads asked for React".

requirements

Column	Type	Meaning
lead_id	leads.id, unique	One requirements record per lead
project_type	web / mobile / both / other	Kind of product
expected_timeline	text	The client's own timeline
budget	numeric	The client's stated budget
notes	text	Anything else
status	pending / received / confirmed	How far the client has got
received_at	timestamp, optional	When the client sent them

requirement_tech_stacks

A link table: one row says "this requirement uses this technology". The primary key is the pair (`requirement_id`, `tech_stack_id`), so a technology cannot be added twice.

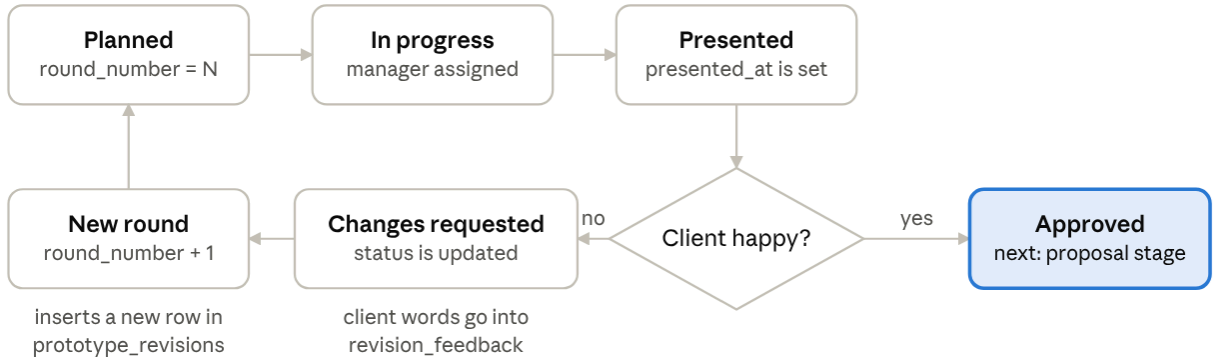
requirement_features

Column	Type	Meaning
requirement_id	requirements.id	Parent requirement
title	text	Feature name
description	text, optional	Detail
priority	must / should / nice	How essential it is

Prototype loop tables

Clients ask for changes again and again after seeing a prototype. Each ask is a new round, so you keep a full history and can see how many rounds a deal has taken, which helps when you negotiate price.

A rejected prototype loops back as a new round, not an edit



prototype loop · one row per round, feedback stored separately

A round never gets edited into the next one. The old row stays as it was and a new row with `round_number + 1` is added.

prototype_revisions

Column	Type	Meaning
lead_id	leads.id	Which lead
round_number	integer	1, 2, 3 and so on. Unique per lead together with lead_id.
type_id	revision_types.id	initial, change request, new feature or bug fix
status_id	revision_statuses.id	planned, in progress, presented, approved or changes requested
description	text	What the client asked for in this round
assigned_to	users.id, optional	Who builds it
deadline	date, optional	Delivery date
prototype_link	text, optional	Figma or demo URL
presented_at	timestamp, optional	When it was shown to the client

revision_feedback

The client's reactions to a round. A round can collect several comments, so this is its own table.

Column	Type	Meaning
revision_id	prototype_revisions.id	Which round
feedback	text	What the client said
recorded_by	users.id	Who wrote it down
received_at	timestamp	When it was received

Meetings tables

A repeating meeting is stored as one rule plus one row per occurrence. Creating "every 2 days, 5 times" writes 1 row in `meeting_series` and 5 rows in `meetings`, all sharing the same `series_id`. Cancelling or moving a single occurrence then never touches the others.

meeting_series

The repeat rule, stored once. One-off meetings have no series.

Column	Type	Meaning
title	text	Series name
repeat_every_days	integer, above 0	Gap between occurrences
repeat_count	integer, above 0	How many occurrences
created_by	users.id	Who set it up

meetings

One row per meeting that appears on the calendar.

Column	Type	Meaning
series_id	meeting_series.id, optional	Empty for one-off meetings

Column	Type	Meaning
lead_id	leads.id, optional	The lead it relates to. Kept as empty if the lead is deleted.
title, description	text	Name and agenda
start_at, end_at	timestamp	Timing. The database rejects an end before the start.
meeting_link	text, optional	Meet or Zoom link
organizer_id	users.id	Who created it
status	scheduled / done / cancelled	Current state

meeting_attendees

Who gets the invite email. `email` is required, `user_id` is filled only for team members, so outside clients can be invited too. The pair (`meeting_id` , `email`) is unique, so nobody is invited twice.

Column	Type	Meaning
meeting_id	meetings.id	Which meeting
user_id	users.id, optional	Set for internal attendees
email, name	text	Where the invite goes
invite_sent_at	timestamp, optional	Set after the email is sent

Support tables

attachments

File references. The file itself sits in Backblaze; the database stores only where to find it.

Column	Type	Meaning
lead_id	leads.id	Which lead the file belongs to

Column	Type	Meaning
revision_id	prototype_revisions.id, optional	Set when the file belongs to one prototype round
uploaded_by	users.id	Who uploaded it
file_name	text	Name shown to users
storage_key	text	Path of the file in Backblaze
size_bytes	bigint, optional	File size

email_log

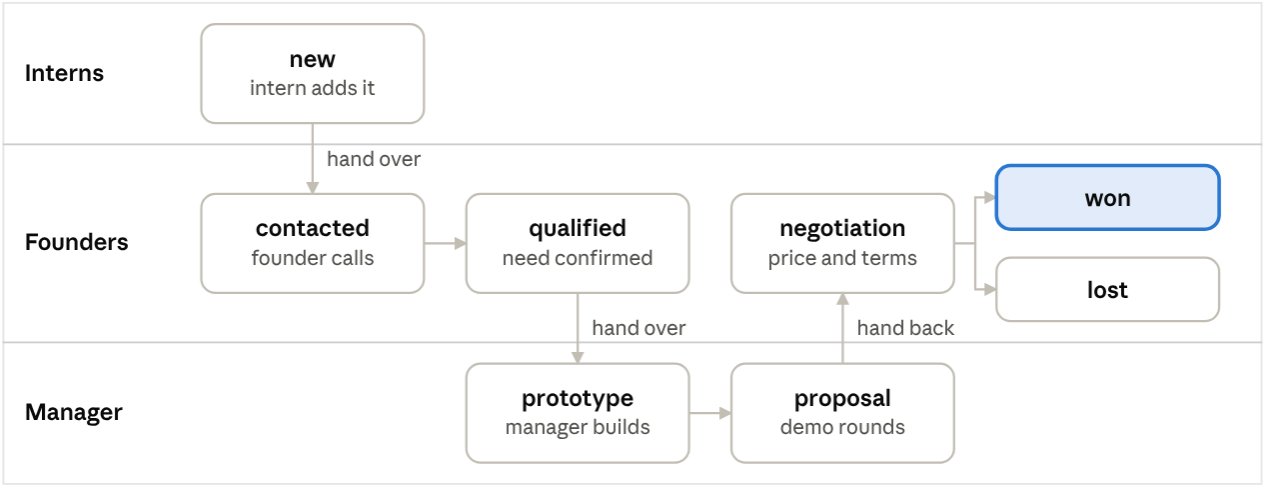
A record of every email the system tries to send (follow-up reminders, meeting invites, handover notices). It answers "I never got the reminder" and stops the same email going out twice.

Column	Type	Meaning
to_email	text	Recipient
type	text	follow_up_reminder, meeting_invite or handover
subject	text, optional	Email subject
status	queued / sent / failed	Result
error	text, optional	Provider's error message when it failed
sent_at	timestamp, optional	When it was sent
lead_id, meeting_id	optional	What the email was about

Lead lifecycle

The stage of a lead decides who owns it. Each lane below is a role, and an arrow that crosses lanes is a handover.

A lead changes owner three times on its way to won or lost



Any stage can end as lost. Every move is logged in lead_stage_history.

lead lifecycle · 8 stages across 3 roles

Moving a lead to a new stage is one transaction that writes four things together, so the data never disagrees with itself:

1. Update `leads.stage_id` and set `leads.stage_changed_at` to now.
2. Insert a row in `lead_stage_history` with the old stage, the new stage and who moved it.
3. If the new stage belongs to a different role (the `owner_role` of the stage), set `leads.assigned_to` to a user with that role.
4. Insert a row in `email_log` and send the handover email to the new owner.

When the stage becomes won or lost, also set `leads.closed_at`. Lost needs a `lost_reason_id`; won should have a `deal_value`.

Access rules

The database does not enforce who sees what. The Django backend does, by adding a filter to every query. These are the rules agreed so far.

Role	Leads they can see	Dashboard	Users and settings
Founder	All leads	Yes, founders only	Manage users, edit lookup lists
Manager	Leads at prototype stage or later, and any assigned to them	No	No
Intern	Leads they created or are assigned	No	No

The filters come from two columns on `leads`: `created_by` and `assigned_to`, plus `stage_id` for the manager rule. Child tables (activities, follow-ups, requirements, revisions, meetings) inherit access from their lead, so each query joins back to `leads` and applies the same filter.

When a lead reaches the `prototype` stage, `assigned_to` becomes the manager and the manager gets an email. When it reaches `negotiation`, it returns to a founder.

How to change things

Because there is no ORM, every change is a SQL statement you run once against `neondb`, plus a matching edit in `backend/schema.sql` so a fresh database stays identical. Run changes inside `BEGIN; ... COMMIT;` so a mistake rolls back.

I want to...	Do this
Add a pipeline stage, source or lost reason	`INSERT` into the lookup table. For a stage, pick a `position` and shift later ones with an `UPDATE`. No code change needed.
Rename a stage or source	`UPDATE` its `name`. Every lead follows automatically because tables store the `id`.
Add a field to leads	`ALTER TABLE leads ADD COLUMN website_note TEXT;` Existing rows get null. Then update the lead form and queries.
Make a new field required	Backfill existing rows first, then `ALTER COLUMN ... SET NOT NULL`. Doing it in the other order fails.
Add a new fixed choice (for example a project type)	Change the `CHECK` list: drop the constraint and add it again with the new value.

I want to...	Do this
Add a fourth role	`INSERT INTO roles`, then decide its rules in the backend filters.
Allow several managers	No schema change. Add a rule that picks which manager gets the handover.
Track projects after a lead is won (phase 2)	Add `projects` and `milestones` tables, each pointing at `leads.id`. Nothing existing needs to change.
Remove a column	Check no query uses it, then `ALTER TABLE ... DROP COLUMN`. This cannot be undone, so export first.

Delete behaviour

Deleting a lead removes its history rows (stage history, activities, follow-ups, requirements, revisions, attachments) because those foreign keys use `ON DELETE CASCADE`. Meetings and emails survive with an empty `lead_id`. Prefer marking a lead lost over deleting it.

Conventions and gotchas

- **Always use parameters in SQL.** With psycopg, write `WHERE id = %s` and pass values separately. Never build SQL by joining strings with user input, which opens the door to SQL injection.
- **Hash passwords.** `users.password` holds a hash, not the password. Never log it or return it from an API.
- **Times are UTC.** Convert to IST only when displaying.
- **Duplicates are not blocked by the database.** Only indexes on `lower(contacts.email)` and `contacts.phone` exist, so the import code must check for duplicates and decide to skip, flag or merge.
- **Lookups are referenced by id, not name.** Join to the lookup table when you need the word, for example `JOIN pipeline_stages ps ON ps.id = leads.stage_id`.
- **`next follow-up` is a query, not a column.** Get it with `MIN(due_at)` from `follow_ups` where status is pending. This avoids a copy that could go out of date.
- **Neon pooled host.** The app uses the `-pooler` connection string. Run schema changes on the direct one.

- **Free tier.** Neon's compute sleeps when idle, so the first query after a quiet period takes about a second longer. Data is not affected.
- **Re-running schema.sql is safe.** It only creates what is missing, but it will not alter an existing column. Column changes need an explicit `ALTER TABLE`.